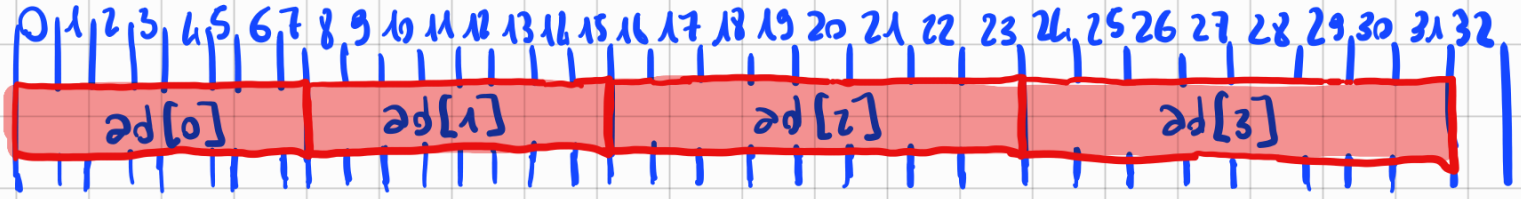
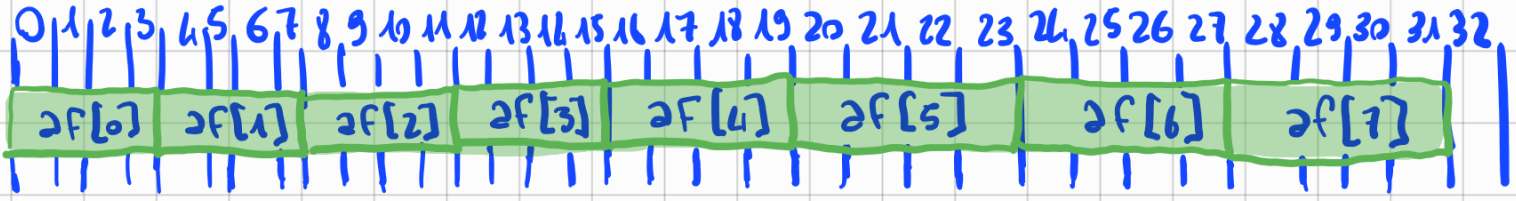
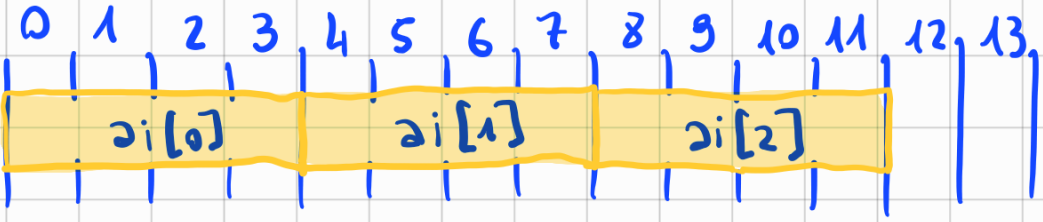
Rappresentiamo graficamente come sono posizionati in memoria gli array. Supponiamo sempre, per semplicità, di cominciare dall'indirizzo 0. L'array "ad" è un array di 4 double, ognuno dei quali occupa 8 byte, cioè 8 indirizzi di memoria:



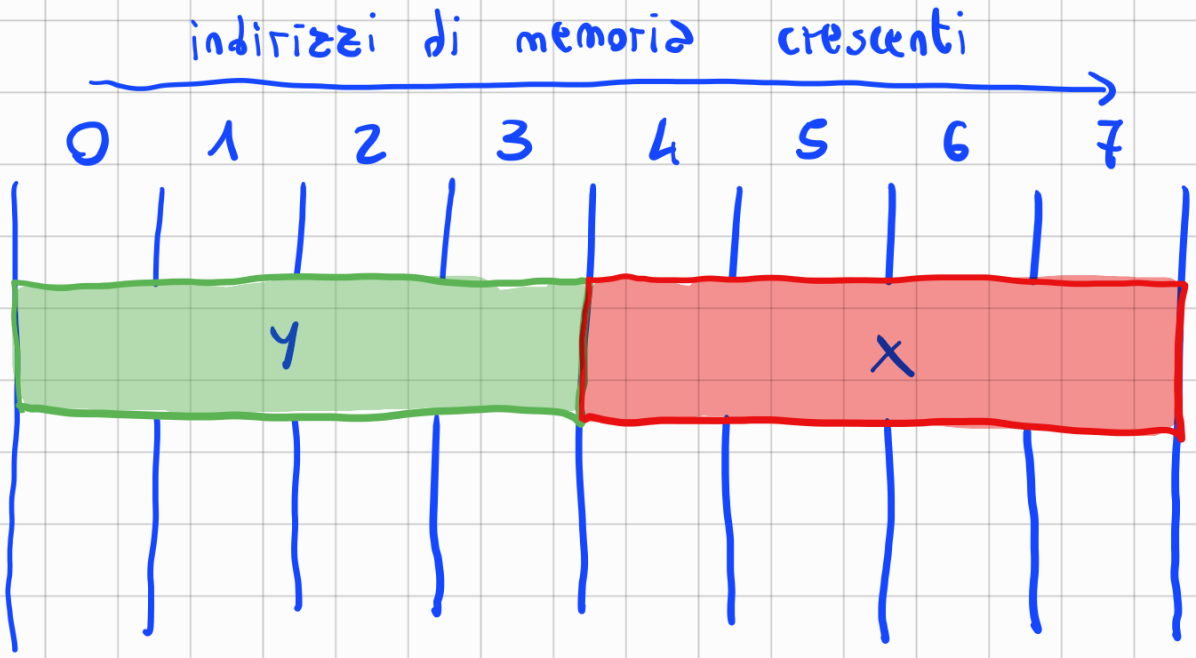
Invece l'array "af" è un array di 8 float, ognuno dei quali occupa 4 byte, ovvero 4 indirizzi di memoria:



Infine, l'array "ai" è un array di 3 interi, ognuno dei quali occupa 4 byte:



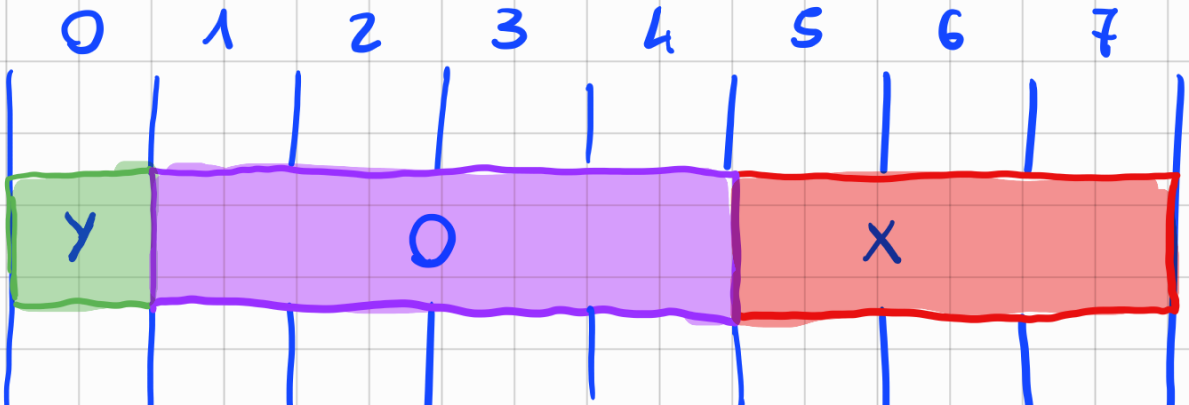
Stampando gli indirizzi di x e y, si vede che x e y sono posizionate presso due indirizzi di memoria adiacenti. In particolare, y è posizionato prima di x. La situazione grafica è la seguente:



Infatti, y è un float e x è un int, quindi entrambi occupano 4 byte, cioè 4 indirizzi di memoria. Quando eseguo il comando:

```
(&y)[1]=0;
```

sto allocando l'intero 0 nel secondo indirizzo di memoria in cui è memorizzato y. Ossia ho una situazione di questo tipo:



Quindi, quando vado a stampare x, viene restituito l'elemento presente nel primo indirizzo di memoria dove x era allocato. Ma questo elemento ora è l'intero 0.
Provando invece a stampare y con il comando:

```
std::cout << y << "\n";
```

si ottiene ancora 1.1.